

Projet IA : Méthodologie et déploiement

Titre du projet : Détecteur de plantes malades

Nom des étudiants :

Mohamed Sanoussy Camara

&

Mohamed Yacine Tarfi

Période : 3 mois

Date : 30 septembre 2025

Membres l'équipe :

- Mohamed Sanoussy Camara
- Mohamed Yacine Tarfi

1. Définition du problème

1.1 Client

Une coopérative agricole regroupe des producteurs de tomates et de pommes de terre. Elle souhaite un outil intelligent pour détecter rapidement les maladies des plantes afin de réduire les pertes de rendement, optimiser l'usage des produits chimiques et accélérer le diagnostic sans expertise humaine immédiate.

1.2 Problème concret à résoudre

Les producteurs peinent à identifier rapidement les maladies affectant leurs cultures, ce qui entraîne :

Baisse de rendement traitements inadaptés ou

tardifs perte de temps pour la surveillance manuelle.

1.3 Causes et contraintes

Causes : difficultés à identifier certaines maladies à l'œil nu, détection tardive, manque de formation spécialisée.

Contraintes : s'appuyer uniquement sur des jeux de données publics (PlantVillage – Kaggle), formats hétérogènes (JPG/PNG, résolutions variables), déséquilibre potentiel de classes

1.4 Conséquences

- Réduction significative du rendement agricole et des revenus des agriculteurs.
- Sur-traitement chimique ou interventions inutiles.
- Perte de temps pour la surveillance manuelle des cultures.

1.5 Objectifs du projet

Qualitatifs : fournir un outil pratique et fiable pour assister les agriculteurs dans la gestion sanitaire de leurs cultures.

Mesurables : atteindre une précision d'identification ≥ 85 % sur les images testées et réduire le temps moyen de diagnostic à moins d'une minute par image.

1.6 Périmètre du projet

- Plantes ciblées : cultures courantes de la coopérative (tomates, pommes de terre)
- Maladies couvertes : les plus fréquentes et impactantes (mildiou, tavelure, taches foliaires, etc.)
- Données : Limitation aux images disponibles via datasets publics

1.7 Solution IA envisagée

Modèle :

Réseau de neurones convolutionnel (CNN) pour classification d'images

Solution :

Application / tableau de bord affichant le diagnostic et des recommandations d'action

2. Collecte de données

Jeu de données principal : PlantVillage (Kaggle ~ 87 k images)

- **Volume** : ~87 000 images RGB

- **Classes** : ~38 (espèce + maladie/sain)

Pertinence : images annotées de feuilles saines et malades, avec identification précise du type de maladie

Contraintes : formats JPG/PNG, résolution variable, déséquilibre possible entre classes

Sources supplémentaires : autres datasets open-source ou API agricoles

3. Préparation des données

Exploration initiale : Nous avons commencé à analyser le dataset pour mieux comprendre sa structure et sa qualité. Cette exploration a inclus :

- Vérification des formats et résolutions des images (JPG/PNG, tailles variables) pour anticiper le prétraitement.
- Comptage du nombre d'images par classe afin de détecter un éventuel déséquilibre entre les maladies et les feuilles saines.
- Observation d'un échantillon représentatif de chaque classe pour vérifier la qualité visuelle des images et la diversité des conditions (luminosité, angle, stade de la maladie).

Identification des problèmes potentiels à corriger : images floues, annotations manquantes ou incorrectes.

Strawberry__healthy



Grape__Black_rot



Potato__Early_blight



Blueberry__healthy



Corn_(maize)__healthy



Tomato__Target_Spot



Peach__healthy



Potato__Late_blight



Tomato__Late_blight



Échantillon de données

	filepath	label	species	disease	format	resolution
0	/Users/yacine/Desktop/archive/New Plant Diseas...	Strawberry___healthy	Strawberry	healthy	JPG	256x256
1	/Users/yacine/Desktop/archive/New Plant Diseas...	Grape___Black_rot	Grape	Black_rot	JPG	256x256
2	/Users/yacine/Desktop/archive/New Plant Diseas...	Potato___Early_blight	Potato	Early_blight	JPG	256x256
3	/Users/yacine/Desktop/archive/New Plant Diseas...	Blueberry___healthy	Blueberry	healthy	JPG	256x256
4	/Users/yacine/Desktop/archive/New Plant Diseas...	Corn_(maize)___healthy	Corn_(maize)	healthy	JPG	256x256
5	/Users/yacine/Desktop/archive/New Plant Diseas...	Tomato___Target_Spot	Tomato	Target_Spot	JPG	256x256
6	/Users/yacine/Desktop/archive/New Plant Diseas...	Peach___healthy	Peach	healthy	JPG	256x256
7	/Users/yacine/Desktop/archive/New Plant Diseas...	Potato___Late_blight	Potato	Late_blight	JPG	256x256
8	/Users/yacine/Desktop/archive/New Plant Diseas...	Tomato___Late_blight	Tomato	Late_blight	JPG	256x256
9	/Users/yacine/Desktop/archive/New Plant Diseas...	Tomato___Tomato_mosaic_virus	Tomato	Tomato_mosaic_virus	JPG	256x256

4. Constat

Le dataset PlantVillage est riche et pertinent mais imparfait (déséquilibre de classes, images bruitées). Un prétraitement rigoureux est indispensable. Le dictionnaire de données sera complété au fur et à mesure de l'analyse exploratoire.

Contrat d'Optimisation (Jalon #2)

Projet : Détecteur de plantes malades - Classification multi-classes par Deep Learning (CNN).

5. Contrat d'optimisation

Notre objectif principal de maximiser l'Accuracy (Précision globale) du modèle sur le jeu de test.

Justification : L'objectif est d'obtenir le plus haut taux de prédictions correctes, afin de garantir un modèle fiable dans la majorité des cas et de valider la faisabilité technique du modèle.

6. Contraintes

L'optimisation sera menée en respectant les contraintes opérationnelles et de déploiement suivantes :

- **Contrainte 1 (Développement)** : Le jeu de données est très volumineux (87 000 images). Par conséquent, l'entraînement d'un seul modèle est une opération longue, ce qui limite le nombre total d'expérimentations que nous pouvons réaliser.
- **Contrainte 2 (Déploiement)** : La taille du modèle final doit être **inférieure ou égale à 100 Mo** (pour un déploiement sur appareil mobile/embarqué).

Métriques de Performance Conformément aux ajustements clés de l'évaluation, nous utiliserons :

- **Métrique Primaire** :
 - **Accuracy (Précision globale)** : La cible d'optimisation principale.

- **Métriques Secondaires :**
 - **Temps d'entraînement (h)** : Surveillé pour garantir le respect de la Contrainte 1.
 - **Taille du modèle (Mo)** : Surveillée pour garantir le respect de la Contrainte 2.

7. Budget de Ressources et de Travail

- **Ressources** : L'optimisation est limitée à un maximum de **15 heures GPU** pour le tuning d'hyperparamètres.
- **Temps de travail** : Nous allouons un maximum de 3 jours d'expérimentation et d'analyse (environ **25 heures** de travail humain).

8. Modélisation

8.1 Stratégie Cohérente et Comparaison (Ajustement Clé #2)

Notre stratégie de modélisation est directement liée aux **contraintes** définies dans notre Contrat d'Optimisation. Nous devons trouver un modèle qui maximise l'**Accuracy** (Objectif) tout en respectant nos contraintes de **Taille**

Pour cela, nous avons comparé deux architectures de la famille ResNet avec des complexités différentes :

1. **ResNet9 (Modèle Léger)** : Choisi car il est reconnu pour son efficacité. Sa petite taille (25.14 Mo) et son temps d'entraînement plus court en font le candidat idéal pour respecter nos deux contraintes.
2. **ResNet18 (Modèle Plus Lourd)** : Choisi pour évaluer si un modèle plus complexe (42.71 Mo) pouvait justifier un **temps d'entraînement plus long** (Contrainte 1) par un gain d'Accuracy significatif.

3.2. Résultats de la Modélisation de Base

L'expérimentation de base a révélé un résultat clair : le modèle **ResNet9** (le plus léger) était non seulement plus rapide à entraîner et plus petit, mais il a aussi surpassé le ResNet18 en performance (Accuracy).

9. Évaluation & Optimisation

9.1 Distinction des Métriques (Ajustement Clé #3)

Comme stipulé dans notre Contrat d'Optimisation, l'évaluation a été guidée par une distinction claire entre nos métriques :

- **Métrique Primaire (Accuracy)** : Elle a servi à mesurer le succès de notre objectif principal.
- **Métriques Secondaires (Taille, Temps d'entraînement)** : Elles ont servi à surveiller le respect de nos contraintes (Contrainte 1 et 2).

4.2. Stratégie d'Optimisation (Ajustement Clé #4)

L'optimisation (Ajustement Clé #4) a été menée sur le modèle gagnant de la phase précédente : le **ResNet9**.

Notre cahier de code ([.ipynb](#)) montre l'utilisation de plusieurs techniques de *tuning* structurées (plutôt que de simples recherches aléatoires) pour stabiliser et améliorer l'entraînement :

- La gestion dynamique du **Taux d'apprentissage** (*Learning Rate Scheduling*).
- Des techniques de régularisation (comme le *Weight Decay*) pour améliorer la généralisation.

4.3. Tableau de Résultats (Base vs Optimisé)

Le modèle ResNet9 de base avait déjà une performance de 99.4%. Les efforts d'optimisation ont donc visé à confirmer ce score élevé tout en s'assurant que les contraintes étaient toujours respectées.

9.2 [TABLEAU DES RÉSULTATS D'OPTIMISATION]

Version du Modèle	Accuracy (Métrique Primaire)	Taille (Contrainte 2)
ResNet9 (Base)	99.4%	25.14 Mo
ResNet9 (Optimisé)	99.6%	25.14 Mo

10. Modèle Final

10.1 Justification du Choix (Ajustement Clé #5)

Le choix final s'est porté sur le **ResNet9 (Optimisé)**.

Ce modèle a parfaitement atteint notre **Objectif Principal** (maximiser l'**Accuracy**), en obtenant un score exceptionnel de **99.4%** sur le jeu de test.

Le modèle respecte également nos deux contraintes définies dans le contrat :

1. **Contrainte 1 (Temps)** : Le modèle de base s'entraîne en ~4 heures, un temps de référence gérable.
2. **Contrainte 2 (Taille)** : Le modèle final a une taille de **25.14 Mo**, ce qui est bien inférieur à notre limite de 100 Mo.

5.2. Sauvegarde (Ajustement Clé #5)

Le modèle final a été sauvegardé au format **.pt** (PyTorch) pour garantir sa reproductibilité et son déploiement futur.

11. API de prédiction

Le modèle final est intégré dans une API développée avec FastAPI. Cette API permet de rendre le modèle accessible via des requêtes HTTP standard, sans exposer directement les détails d'implémentation du modèle.

L'API expose un endpoint de santé permettant de vérifier la disponibilité du service et le bon chargement du modèle, ainsi qu'un endpoint de prédiction acceptant une image au format multipart/form-data.

Une interface Swagger est automatiquement générée et accessible via l'endpoint /docs. Elle permet de tester les prédictions directement grâce à la fonctionnalité Try it out.

12. Conteneurisation avec Docker

L'API est conteneurisée à l'aide de Docker afin de garantir la portabilité et la reproductibilité de l'environnement d'exécution. Le conteneur inclut l'ensemble des dépendances nécessaires au fonctionnement du modèle et de l'API.

Le serveur Uvicorn est lancé à l'intérieur du conteneur et écoute sur le port défini par la variable d'environnement PORT, conformément aux contraintes de Google Cloud Run.

13. Déploiement sur Google Cloud Run

Le déploiement est réalisé sur Google Cloud Run, une plateforme serverless permettant un démarrage à la demande et une montée en charge automatique. L'image Docker est construite via Cloud Build puis déployée à partir d'Artifact Registry.

Une attention particulière est portée à la configuration des comptes de service IAM afin de garantir le bon démarrage du service et l'accès aux ressources nécessaires.